

Lab 1 – software engineering

Name – Krishna Keerthan Reddy D.R

Class – D

Section – PES1UG24CS237

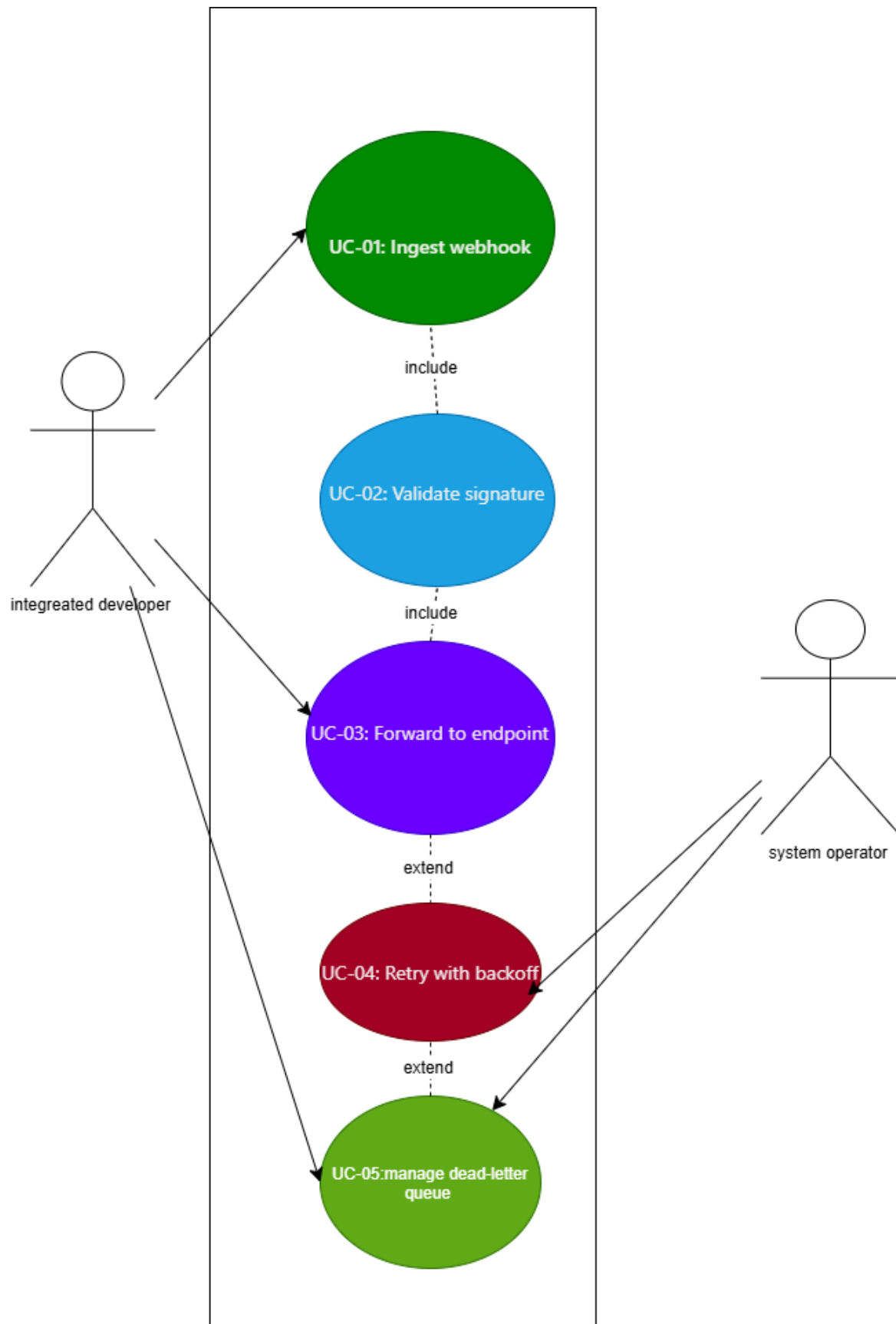
Problem statement-Webhook Ingestion & Retry Mechanism Hub

Deliverable one-requirement table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall ingest inbound webhooks, forward the payload to the target endpoint, and automatically retry failed deliveries using exponential backoff (1s, 2s, 4s, 8s).	High	Pass: a failed delivery is retried at the correct intervals and logged each time. Fail: a webhook is dropped and never shows up in the dead-letter queue.	This is basically the core of the whole system, without this nothing else matters.
FR-002	Functional	Before forwarding anything, the system must check the HMAC signature on the incoming request and reject it (401) if it doesn't match.	High	Pass: bad signature = 401, request not forwarded. Fail: a request with no/incorrect signature still gets forwarded to an internal service.	Without this literally anyone could POST fake events to us.
FR-003	Functional	Every webhook the system receives shall be logged in full (headers + body + timestamp + source IP) and kept for at least 30 days.	High	Pass: can pull up any event from the last 30 days with all fields intact. Fail: an event is missing or some field got dropped.	Needed for debugging + if we ever need to prove an event was/wasn't received.
FR-004	Functional	If a webhook fails all of its retry attempts, the system shall move it into a dead-letter queue and let the System Operator know within 5 minutes.	Medium	Pass: after 4 failed retries the event lands in the DLQ and an alert goes out in under 5 min. Fail: it just disappears, or the alert takes forever.	Ops needs to know when something's actually broken, not just silently lose events.
FR-005	Functional	The system shall let an Integration Developer manually re-send (replay) a specific webhook from the dead-letter queue through an API call.	Medium	Pass: calling the replay endpoint on a DLQ item re-triggers delivery and removes it from the DLQ once it succeeds. Fail: nothing happens, or it stays stuck in the DLQ.	So devs aren't stuck waiting on us if the downstream service was just temporarily down.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Nonfunc.	The ingestion endpoint shall buffer inbound webhooks with 99.999% reliability and handle up to 2,500 requests/second.	High	Pass: benchmark test at 2,500 req/s for 10 min shows no dropped requests. Fail: any requests get dropped under that load.	Given as part of the problem statement, kept as-is.
NFR-002	Nonfunc.	End-to-end delivery time (from receiving the webhook to forwarding it) should stay under 500ms for 95% of requests under normal traffic.	High	Pass: p95 latency stays at or under 500ms over a 30-min test at 1000 req/s. Fail: p95 goes above 500ms.	If we're too slow, downstream services might time out waiting on us.

Deliverable two -UML USE CASE DIAGRAM



Deliverable three - Use-Case Flow Specification

Actors

Integration Developer (registers the endpoint, can replay from the DLQ later), System Operator (gets notified if things go wrong).

Preconditions

- The gateway service is up and reachable from the internet.
- The third-party sender already has our webhook URL configured on their end.
- We already have the HMAC secret set up for that source (assumed done during onboarding, not part of this flow).

Postconditions

- The payload has been written to the request log.
- The payload has been forwarded to whatever internal service it was meant for.
- Caller gets a 200 OK back.

Main Success Scenario

- 1. Third-party service POSTs the event to our ingestion endpoint, payload is signed.
- 2. Gateway responds with 200 right away so the sender doesn't time out waiting on us.
- 3. Gateway checks the HMAC signature against the secret we have on file for that source.
- 4. Signature checks out, so the full payload (headers, body, timestamp, source IP) gets written to the request log.
- 5. Gateway forwards the payload to the correct internal endpoint.
- 6. Internal service responds 2xx.
- 7. Delivery marked successful, use case ends.

Alternate Flow - internal service doesn't respond properly (branches off step 6)

- A1. Delivery gets marked failed.
- A2. Retry gets scheduled with backoff (1s, then 2s, 4s, 8s) - this is basically handing off to UC-04.
- A3. If it still fails after all retries, event goes into the dead-letter queue (UC-05).
- A4. System Operator gets pinged within 5 min so someone actually notices.

